

Verbesserungen 1 — Dashboard, Reichweiten, Gegner-KI, Bosse, Quests

Änderungsliste zum bereits gebauten Spiel. Jeder Abschnitt ist **in sich abgeschlossen** und kann einzeln umgesetzt werden — auch von einer KI, die den restlichen Code nicht kennt.

Grundlage sind acht Punkte aus dem Spieltest. Werte, die hier stehen, ersetzen die bisherigen; alles Unerwähnte bleibt wie es ist.

#	Was	Abschnitt
1	Dashboard umbauen: Spielerbild in der Mitte, Kacheln außenrum	1
2	Reichweiten begrenzen (Bogen, Speer, Gegner-Geschosse)	2
3	Sumpf entzerren, Krokodile sollen schwimmen	3
4	Wasser besser animieren	4
5	Gegner auf höheren Stufen schneller und schlauer	5
6	Titanoboa deutlich stärker	6
7	Ork-Häuptling stärker	7
8	Mehr Quests in drei Schwierigkeitsklassen	8

1. Dashboard-Umbau

Das Dashboard wird von drei großen Bereichen auf ein **3×3-Raster** umgestellt. In der Mitte steht der Held, außen herum liegen die Kacheln.

CHARAKTER	QUESTS	AUSRÜSTUNG
Werte·Skills	3 aktive Ziele	2 von 3 Waffen
SHOP		SCHMIED
Waffen·Schild	SPIELER-	Schärfung
Rüstung·Tränke	BILD	Trankgürtel
	Held · Stufe 13	
	[ XP]	
	4.820 G · 2 SP	
STATISTIKEN	PFAD	EINSTELLUNGEN
Kills·Tode	Weiter: Sumpf	Ton·Neues
Gold·Zeit	[SPIELEN]	Spiel

Die Mitte: das Spielerbild

Ein großes Pixel-Bild des Helden in Ganzkörperansicht, **mit der aktuell ausgerüsteten Ausrüstung**: Wer den Stahlspeer und die Plattenrüstung trägt, sieht das im Bild. Das ist der Grund, warum die Mitte funktioniert — sonst wäre es nur Dekoration. Es macht jedes Upgrade sofort sichtbar, noch bevor man ins Level geht.

Für den Anfang reicht eine einfache Umsetzung: übereinandergelegte PNG-Schichten (Körper → Rüstung → Waffe), jeweils dasselbe Bildmaß. Später austauschbar, ohne den Code zu ändern.

Darunter: Name, Stufe, XP-Leiste, Gold, freie Skillpunkte.

Die acht Kacheln

Position	Kachel	Inhalt
oben links	Charakter	Werte, Skillpunkte verteilen, Skillbaum
oben mitte	Quests	die 3 aktiven Quests mit Fortschrittsbalken
oben rechts	Ausrüstung	Waffenwahl: 2 von 3 mit ins Level
mitte links	Shop	Schwert, Schild, Bogen, Speer, Rüstung, Tränke
mitte rechts	Schmied	Waffenschärfung, Trankgürtel
unten links	Statistiken	Kills nach Typ, Tode, Gold gesamt, Spielzeit
unten mitte	Pfad	Vorschau der Route, nächstes Level, Spielen-Knopf
unten rechts	Einstellungen	Lautstärke, Neues Spiel, Steuerungsübersicht

Verhalten der Kacheln

Jede Kachel zeigt eine **Kurzfassung** und öffnet bei Klick ein großes Fenster (Overlay) über dem Dashboard. Die Shop-Kachel zeigt also z. B. nur „3 Upgrades verfügbar · 1 bezahlbar“, das Detail kommt im Overlay. Sonst wird das Raster unlesbar voll.

Die Pfad-Kachel zeigt die Route als kleine Vorschau mit dem nächsten Level hervorgehoben und einem großen **SPIELEN**-Knopf. Klick auf die Kachel öffnet die volle Route mit allen Knoten, Sternen und der Schwierigkeitswahl.

Hinweise, die auffallen müssen:

- Freie Skillpunkte → Charakter-Kachel pulsiert
- Abholbereite Quest-Belohnung → Quest-Kachel pulsiert
- Neu bezahlbares Upgrade → kleiner Punkt auf der Shop-Kachel

Ohne diese Hinweise übersieht man in einem 8-Kachel-Raster ständig etwas.

Technisch

CSS-Grid, drei Spalten und drei Zeilen, mittlere Spalte etwas breiter:

```
grid-template-columns: 1fr 1.4fr 1fr;
```

```
grid-template-rows: 1fr 1.5fr 1fr;
```

Bleibt HTML/CSS wie bisher, kein Canvas. Bei schmalen Fenstern (unter 1100 px) brechen die Kacheln auf zwei Spalten um, das Spielerbild rutscht nach oben.

2. Reichweiten begrenzen

Bisher fliegen Geschosse unbegrenzt weit — dadurch kann der Spieler Gegner abschießen, die er kaum sieht, und wird von Gegnern getroffen, die außerhalb des Bildes stehen. Beides fühlt sich falsch an.

Alle Werte in Pixeln; eine Kachel = 32 px.

Waffe / Angriff	Reichweite	Kacheln
Schwert (Hieb)	40 px	1,25

Waffe / Angriff	Reichweite	Kacheln
Speer (Stoß)	60 px	~2
Speer (Wurf)	320 px	10
Bogen (Pfeil)	480 px	15
Gorilla (Stein)	288 px	9
Bogenschütze (Pfeil)	352 px	11
Titanoboa (Giftspucke, Abschnitt 6)	400 px	12,5

Die Reihenfolge ist Absicht: Der Bogen des Spielers reicht weiter als jedes Gegner-Geschoss. Das ist der ganze Sinn der Waffe — sie kauft Sicherheit gegen wenig Schaden. Wenn ein Bogenschütze genauso weit schießt, hat der Bogen keine Existenzberechtigung mehr. Der Speerwurf liegt bewusst deutlich darunter.

Umsetzung:

- Jedes Geschoss merkt sich seinen Startpunkt und verschwindet, sobald die Reichweite überschritten ist.
- **Sichtbar verschwinden, nicht einfach ausblenden:** Pfeile bleiben kurz im Boden stecken und verblassen, Steine zerspringen. Sonst wirkt es wie ein Fehler.
- Gegner mit Fernkampf **beginnen erst zu zielen, wenn der Spieler in Reichweite ist**, und hören auf, wenn er sie verlässt. Kein Schießen ins Leere.
- Reichweite des Wahrnehmens = $1,3 \times$ Angriffsreichweite. So läuft der Spieler nicht in eine Gruppe, die ihn schon von weitem beschießt.

Alle Werte gehören in `config.js` als `range` pro Waffe und pro Monster.

3. Sumpf entzerren, Krokodile schwimmen

Aus dem Spieltest: Neun Krokodile im Sumpf sind zu eng, sie kommen sich gegenseitig in die Quere.

Wasser als neuer Kacheltyp

Ein dritter Kacheltyp neben Boden und Wand:

Kacheltyp	Spieler	Krokodil	Andere Gegner	Geschosse
Boden	✓	✓	✓	✓
Wand	✗	✗	✗	✗ (blockiert)
Wasser	✗	✓ (schwimmend)	✗	✓ (fliegen darüber)

Das Krokodil bewegt sich im Wasser mit **200 px/s** statt 120 — schneller als der Spieler an Land. Dadurch wird Wasser zu seinem Revier: Der Spieler kann nicht hinein, das Krokodil kommt jederzeit heraus. Genau das macht den Sumpf zu einem eigenen Level statt zu „Wiese mit anderen Gegnern“.

Im Wasser ist das Krokodil **nicht angreifbar** — dieselbe Regel wie beim Abtauchen. Der Schatten bzw. die Kielwelle bleibt sichtbar.

Neues Sumpf-Layout

- **Karte vergrößern** auf 60×40 Kacheln (statt 40×30), damit Platz entsteht.
- **Drei Wasserflächen** mit trockenen Wegen und Inseln dazwischen.
- **Krokodile in drei Gruppen zu dritt**, je eine pro Wasserfläche, statt neun auf einmal. Die nächste Gruppe wird erst aktiv, wenn der Spieler ihren Bereich betritt.
- Mindestens 4 Kacheln freier Boden zwischen den Wasserflächen, damit man kämpfen kann, ohne in die Enge zu geraten.

Anzahl und Werte der Krokodile bleiben unverändert — es ist ein Platzproblem, kein Balanceproblem.

4. Wasser animieren

Vier Ebenen, von unten nach oben — jede für sich einfach, zusammen überzeugend:

1. **Kachel-Animation:** 4 Bilder im Wechsel, 0,4 s pro Bild. Reicht schon für den Grundeindruck.
2. **Versetzte Startzeiten:** Jede Wasserkachel beginnt ihre Animation mit einem zufälligen Versatz von 0–0,4 s. Ohne das pulsiert die ganze Fläche im Gleichtakt und sieht nach Kacheln aus statt nach Wasser. **Das ist der wichtigste Punkt und kostet eine Zeile.**
3. **Ufer-Schaum:** Wasserkacheln, die an Boden grenzen, bekommen eine eigene Kachelvariante mit heller Schaumkante. Macht optisch den größten Unterschied.

4. **Reaktion auf Bewegung:** Ein Krokodil im Wasser zieht eine **Kielwelle** hinter sich her (zwei bis drei sich ausbreitende helle Bögen). Beim Auftauchen ein **Spritzer** aus 6–8 Partikeln.

Punkt 4 erfüllt zwei Zwecke gleichzeitig: Es sieht gut aus **und** es zeigt dem Spieler, wo das Krokodil gerade ist. Die Kielwelle ist damit kein Schmuck, sondern Teil der Fairness-Regel aus Abschnitt 5.

5. Gegner-KI nach Schwierigkeitsstufe

Bisher unterscheiden sich die Stufen nur durch Zahlen. Ab jetzt auch durch Verhalten. **Was sich nicht ändert: Die Ausholphase bleibt auf allen Stufen bei mindestens 0,4 Sekunden.** Schneller reagieren müssen ist kein Schwierigkeitsgrad.

Normal — unverändert

Gegner laufen direkt auf den Spieler zu und greifen an, sobald sie in Reichweite sind. Jeder für sich.

Schwer — Umzingeln und versetztes Angreifen

Umzingeln: Der Kreis um den Spieler wird in Plätze aufgeteilt (bei drei Gegnern drei Plätze zu je 120°). Jeder Gegner belegt einen freien Platz und nähert sich von dort. Ergebnis: Sie kommen aus verschiedenen Richtungen statt alle als Traube von vorn.

Versetztes Angreifen: Es dürfen **höchstens zwei Gegner gleichzeitig** in der Ausholphase sein. Die anderen warten 0,5–1,0 s (zufällig), dann ist der Nächste dran. Das klingt nach einer Erleichterung, ist aber das Gegenteil: Statt einer Massensalve, die man einmal blockt, kommt ein Dauerdruck, bei dem man ständig nachjustieren muss.

Schild umgehen: Nahkampfgegner bevorzugen einen Platz **außerhalb des 120°-Blockwinkels** des Spielers. Wer stumpf blockt und stehen bleibt, wird von der Seite getroffen. Das ist der eigentliche Sprung von Normal auf Schwer.

Alptraum — zusätzlich Timing gegen den Spieler

Alles von Schwer, plus:

Ausweichrolle abwarten: Erkennt ein Gegner, dass der Spieler gerade ausweicht, verzögert er seinen Angriff um 0,3 s — genau in den Moment, in dem der Spieler aus der Rolle kommt und noch nicht wieder blocken kann. Wer die Rolle als Allzweckmittel benutzt, wird bestraft; wer sie sparsam einsetzt, kommt durch.

Dazu wie bisher: +15 % Bewegungstempo.

Umsetzung: Die drei Verhaltensweisen als Schalter in `config.js` (`surround`, `staggerAttacks`, `punishDodge`), pro Schwierigkeitsstufe an oder aus. So kann man sie einzeln testen und einzeln abschalten, wenn eine sich falsch anfühlt.

6. Titanoboa — deutlich stärker

Wert	Vorher	Neu
HP	500	750
Grundschaden	50	60
XP	500	800
Gold	300 G	500 G

Angriffswahl statt fester Reihenfolge

Der eigentliche Unterschied zwischen „stark“ und „schlau“: Die Boa spielt kein festes Muster ab, sondern **wählt ihren Angriff nach dem Abstand zum Spieler**.

Abstand	Angriff	Schaden
unter 100 px	Schwanzfeger — 360°, Radius 128 px	70
100–350 px	Biss — kurzer Vorstoß	60
über 350 px	Giftspucke — 3 Geschosse, Reichweite 400 px	40 je Treffer

Damit funktioniert keine einzelne Strategie mehr: Wer auf Abstand bleibt, bekommt Giftspucke; wer klebt, bekommt den Schwanzfeger. Der Spieler muss den Abstand aktiv steuern. Das ist mit wenigen Zeilen umsetzbar und wirkt deutlich intelligenter als jede Zufallsauswahl.

Phase 1 (750 – 375 HP)

Wie bisher das Verschlingen aus dem Boden — **100 Schaden**, Schatten wandert sichtbar, bleibt 1,5 s stehen und pulsiert, bevor sie hochschießt. Diese Vorwarnung bleibt unverändert; bei 100 Schaden ist sie Pflicht. Dazwischen die Angriffswahl nach Abstand.

Häutung (bei 375 HP) — jetzt immun

- **3 Sekunden lang immun**, keine Schadensannahme
- Bildschirm wackelt leicht, die alte Haut platzt sichtbar ab
- Sie wächst dabei sichtbar

Ein Vorschlag dazu, den du ignorieren kannst: Bisher war die Häutung das offene Fenster, in dem der Spieler für die halbe Lebensleiste belohnt wurde. Mit Immunität wird daraus eine Zwangspause — der Spieler wird für Fortschritt ausgebremst statt belohnt. Kompromiss: 3 s immun wie von dir gewünscht, **danach 1,5 s Erschöpfung**, in der sie 50 % mehr Schaden nimmt. Die Häutung bleibt unantastbar, der Belohnungsmoment kommt trotzdem — nur eben danach und als Chance, nicht geschenkt.

Phase 2 (unter 375 HP)

- **+20 Schaden** auf alles (Schwanzfeger 90, Biss 80, Spucke 60)
- **30 % schneller** in Bewegung und Angriffsfolge
- Größere Trefferfläche — auch leichter zu treffen
- **Taucht nicht mehr ab**, kämpft offen
- Angriffswahl nach Abstand bleibt, aber die Pausen zwischen den Angriffen sinken von 2,0 s auf 1,4 s

7. Ork-Häuptling — stärker, drei Phasen

Wert	Vorher	Neu
HP	400	600
Grundschaden	25	35
XP	300	450
Gold	200 G	350 G

Auch hier: **Angriffswahl nach Abstand** statt fester Reihenfolge.

Abstand	Angriff	Schaden
unter 80 px	Axtschlag, weiter Schwung, 0,6 s Ausholphase	35
150–400 px	Ansturm — er hält an, eine rote Linie zeigt 1,2 s lang die Bahn, dann stürmt er durch	50
über 400 px	Kriegsruf — ruft 2 Goblins (max. 4 gleichzeitig)	—

Der Ansturm ist der interessanteste Teil: Er ist ausweichbar, wenn man zur Seite geht, aber tödlich, wenn man rückwärts läuft. Das bringt dem Spieler etwas bei, das er im ganzen Spiel gebrauchen kann.

Drei Phasen:

Phase	HP	Verhalten
1	600 – 400	Axtschlag und Ansturm
2	400 – 200	dazu Kriegsruf, alle 12 s bis zu 2 Goblins
3	unter 200	Wutmodus: +30 % Tempo, Axtschlag wird zum Doppelschlag (2× 35), kein Kriegsruf mehr

Der Wutmodus beginnt mit einem sichtbaren Brüllen und 1 s Stillstand — das ist die Vorwarnung, dass es jetzt anders wird.

8. Quests in drei Schwierigkeitsklassen

Freischaltung bleibt **nach Abschluss von Abschnitt 6 (Urwald)**. Vorher ist die Quest-Kachel grau mit dem Hinweis „Wird im Urwald freigeschaltet“.

Neue Regel für die drei Slots

Es sind weiterhin genau **3 Quests gleichzeitig aktiv** — aber ab jetzt **immer eine aus jeder Klasse**:

- **Leicht** (grün) — in ein paar Minuten zu schaffen, oft in alten Leveln
- **Mittel** (gelb) — ein bis zwei Level Arbeit
- **Schwer** (rot) — ein Ziel für mehrere Sitzungen

Wird eine abgeschlossen, rückt die nächste **derselben Klasse** nach. So hat der Spieler immer gleichzeitig etwas Erreichbares und etwas zum Hinarbeiten — statt drei schwerer Quests, bei denen sich stundenlang nichts bewegt.

Die Belohnungen sind bewusst weit auseinander. Leichte Quests sollen sich lohnen, aber niemanden vom eigentlichen Spiel abhalten.

Leichte Quests (grün)

Quest	Belohnung
Besiege 20 Slimes	60 G + 100 XP
Besiege 15 Goblins	80 G + 120 XP
Sammle 300 Gold	80 G + 100 XP
Blocke 15 Angriffe	70 G + 120 XP
Besiege 10 Gegner mit dem Bogen	90 G + 150 XP
Spiele einen der Abschnitte 1–5 noch einmal durch	100 G + 150 XP
Kaufe ein beliebiges Upgrade	50 G + 100 XP

Mittlere Quests (gelb)

Quest	Belohnung
Besiege 15 Gorillas	300 G + 500 XP
Schaffe den Teich	350 G + 600 XP
Besiege 20 Frösche	400 G + 700 XP

Quest	Belohnung
Sammle 2.000 Gold	500 G + 400 XP
Besiege 25 Giftpilze	500 G + 800 XP
Schaffe ein Level, ohne vergiftet zu werden	600 G + 900 XP
Besiege 20 Gegner mit dem Speer	600 G + 800 XP
Erreiche Stufe 12	700 G + 1 Heiltrank

Schwere Quests (rot)

Quest	Belohnung
Besiege 15 Krokodile	1.500 G + 2.000 XP
Schaffe ein Level auf Alptraum	1.800 G + 2.500 XP
Besiege den Ork-Häuptling auf Schwer	2.000 G + 3.000 XP
Besiege die Titanoboa	2.500 G + 3.500 XP
Besiege die Titanoboa auf Alptraum	3.000 G + 4.000 XP
Schaffe alle Level auf Alptraum, jedes ohne zu sterben	3.000 G + 5.000 XP + 5 Skillpunkte

Die letzte Quest behält alle bisherigen Regeln: jeder Tod zählt (auch Wiederbelebung gegen Gold), Fortschritt wird pro Level einzeln gespeichert, Anzeige als Zähler **Alptraum ohne Tod 7/10**.

Die ganze Liste steht in `config.js` mit einem Feld `schwierigkeit: "leicht" | "mittel" | "schwer"`. Neue Quests hinzufügen heißt: eine Zeile ergänzen, kein Code.

9. Umsetzungsreihenfolge

Vom Sichersten zum Riskantesten. Jeder Schritt ist einzeln testbar:

1. **Reichweiten begrenzen** (Abschnitt 2) — kleinste Änderung, größter sofort spürbarer Effekt. Guter erster Test für die Zusammenarbeit mit einer neuen KI.

2. **Wasser-Kacheltyp und Animation** (Abschnitt 4)
 3. **Sumpf neu bauen, Krokodile schwimmen lassen** (Abschnitt 3)
 4. **Gegner-KI: Umzingeln und versetztes Angreifen** (Abschnitt 5, Stufe Schwer)
 5. **Gegner-KI: Ausweichrolle abwarten** (Abschnitt 5, Stufe Alptraum)
 6. **Ork-Häuptling überarbeiten** (Abschnitt 7) — der einfachere der beiden Bosse
 7. **Titanoboa überarbeiten** (Abschnitt 6)
 8. **Quest-Klassen und neue Liste** (Abschnitt 8)
 9. **Dashboard-Umbau** (Abschnitt 1) — zuletzt, weil es die größte Umbaumaßnahme ist und alle anderen Kacheln erst fertig sein sollten
-

10. Arbeiten mit chat.mistral.ai (Le Chat) über GitHub

Le Chat kann per GitHub-Connector direkt auf das Repository zugreifen — Dateien lesen, ändern und Commits bzw. Pull Requests anlegen. Damit entfällt das Kopieren von Hand.

Was Le Chat trotzdem nicht kann: das Spiel starten und selbst ausprobieren. Es sieht den Code, aber nicht das Ergebnis. Testen bleibt deine Aufgabe — und genau darauf sollte der Ablauf ausgelegt sein.

Vorbereitung

1. Projekt auf GitHub pushen, falls noch nicht geschehen.
2. **Dieses Dokument mit ins Repository legen** (**VERBESSERUNGEN_1.md** im Hauptordner). Dann kannst du dich im Chat einfach darauf beziehen, statt Anforderungen jedes Mal neu zu tippen.
3. Den GitHub-Connector in Le Chat mit dem Repository verbinden.

Ablauf pro Änderung

Ein Abschnitt pro Auftrag, ein Branch pro Abschnitt. Nicht mehrere gleichzeitig — sonst weißt du bei einem Fehler nicht, welche Änderung ihn verursacht hat.

Lies **VERBESSERUNGEN_1.md** im Repository. Setze **nur Abschnitt 2 (Reichweiten begrenzen)** um. Lege dafür einen neuen Branch **feature/reichweiten** an.

Alle Zahlenwerte gehören in **src/config.js**. Sag mir vorher, welche Dateien du ändern wirst, und zeig mir danach eine Zusammenfassung der Änderungen. Ändere nichts, was nicht in Abschnitt 2 steht.

Danach:

1. **Branch lokal auschecken und im Browser testen.** Das ist der Schritt, den dir keine KI abnimmt.
2. **Diff auf GitHub anschauen**, bevor du mergst — auch wenn du nicht jede Zeile verstehst, siehst du sofort, ob Dateien angefasst wurden, die gar nichts mit der Aufgabe zu tun haben. Das ist die häufigste Art, wie eine KI ohne Testmöglichkeit Schaden anrichtet.
3. Passt es: mergen. Passt es nicht: Branch verwerfen, nichts kaputt.

Der Branch-Ablauf ist hier wichtiger als bei einem Agenten mit Testzugriff. Wer nicht selbst nachsehen kann, ob etwas funktioniert, sollte wenigstens jederzeit zurückkönnen.

Was gut und was weniger gut funktioniert

Gut geeignet — klar abgegrenzt, wenige Dateien, sofort im Spiel sichtbar:

- Abschnitt 2 (Reichweiten) — bester Einstieg
- Abschnitt 4 (Wasser-Animation)
- Abschnitt 8 (Quest-Liste, größtenteils `config.js`)
- Abschnitt 6 und 7 (Bosse) — je eine Datei, klar beschriebene Werte

Schwieriger — mehrere Dateien gleichzeitig, viel Zusammenspiel:

- Abschnitt 1 (Dashboard-Umbau) — HTML, CSS und mehrere JS-Dateien auf einmal
- Abschnitt 3 (Sumpf) — neuer Kacheltyp, Kollision, Gegner-Bewegung, Level-Datei
- Abschnitt 5 (Gegner-KI) — greift in das Verhalten aller Monster ein

Diese drei gehen auch mit Le Chat, brauchen aber kleinere Teilaufträge: beim Dashboard zum Beispiel erst das leere 3×3-Raster, dann Kachel für Kachel füllen.

Tipp

Bitte Le Chat am Ende jedes Auftrags um einen Satz, **was du im Spiel prüfen sollst**, um zu sehen ob es geklappt hat. Bei den Reichweiten wäre das etwa: „Stell dich weit weg von einem Gorilla — er darf nicht mehr werfen. Schieß einen Pfeil ins Leere — er muss nach 15 Kacheln im Boden stecken bleiben.“ Das ersetzt den Test, den die KI selbst nicht machen kann.